A Reproducible Live Testbed for Ransomware Detection from Server-Side File-Operation Logs: Implementation and Cost-Sensitive Evaluation of Five Machine-Learning Classifiers

[AUTHOR NAME] — *[AUTHOR AFFILIATION, City, Country]*
[AUTHOR EMAIL]

Abstract—Ransomware remains one of the most damaging classes of malicious software, and detecting it early from cheap, privacy-preserving telemetry is an important operational goal. This paper reports an independent, fully reproducible implementation and evaluation of a server-side ransomware-detection method that uses only three aggregate file-operation features — the counts of file **creates**, **renames**, and **deletes** within a one-second window. We reproduce the method's five-classifier comparison (Decision Tree, Random Forest, Support Vector Machine, AdaBoost, and XGBoost), its grid-searched hyperparameter tuning that maximises recall, and its cost-sensitive selection rule in which a missed ransomware event (false negative) is charged ten times a false alarm (false positive). Beyond reproducing the offline pipeline, we build an executable testbed that connects a live file-system observer to the trained classifier, logs every decision to a database, and exposes the state through a REST API and a web dashboard. On the synthetic, balanced dataset used by the testbed, four of the five classifiers attain perfect separation (accuracy, precision, recall, and F1 of 1.000 and total cost 0), while the calibrated support-vector machine achieves 0.975 recall at a total cost of 10. We argue explicitly that these perfect numbers are an artefact of the synthetic generator, whose benign and ransomware windows are linearly separable by construction, and therefore they demonstrate pipeline correctness rather than real detection capability. The contrast with the source method is instructive: evaluated on realistic, imbalanced telemetry from a virtualized small enterprise and real ransomware samples, that method's best classifier (XGBoost) reached a sensitivity of 91.87%, not 100%. The contributions are a reproducible baseline, a transparent cost-sensitive evaluation, and an honest characterisation of the evaluation gap that must be closed with real file-operation logs before any deployment claim can be made.

Keywords—ransomware detection, file-operation logs, cost-sensitive learning, machine learning, real-time monitoring, reproducibility

I. INTRODUCTION

Ransomware encrypts or destroys a victim's files and then demands payment for their release. Because the damage is often irreversible once encryption completes, detection is most valuable in the first seconds of an attack, when the encryption routine is still creating, renaming, and deleting files. Endpoint agents can observe this activity, but they are intrusive and platform-specific. Server-side file-operation logs offer an attractive alternative: they are already produced by the operating system, they contain only metadata (file names and the type of operation), and they do not require inspecting file contents, which keeps them comparatively privacy-friendly and inexpensive.

The research question addressed here is deceptively simple: *can ransomware-like activity be distinguished from ordinary user activity using nothing more than the number of create, rename, and delete operations observed in a short time window?* The method being reproduced answers yes and compares five common classifiers under a cost function that treats a missed attack as ten times more expensive than a false alarm [1]. We implement that method end to end, run it, and report the results honestly.

This paper makes the following contributions.

1. **A reproducible implementation.** We provide an Open-source pipeline that generates the dataset, tunes and trains five classifiers with grid search, persists the resulting artefacts, and selects a best model by the cost rule. The experiment is reproducible from a single command and is exercised by a continuous-integration job on two Python versions.
2. **A live testbed.** Unlike a purely offline study, our system connects a real-time file-system observer to the trained classifier, aggregates events into one-second windows, stores every prediction in a database, and presents live classifications, per-window feature counts, confusion matrices, and cost comparisons through a web dashboard.
3. **A transparent cost-sensitive evaluation.** We report accuracy, precision, recall, F1, ROC AUC, total cost, latency, and training time for every model, and we show that the widely used accuracy metric is uninformative on this task.
4. **An honest gap analysis.** We demonstrate analytically and empirically *why* the synthetic results are perfect — the two classes are separable by a single count threshold — and we specify the experiments that are required before any real-world claim can be supported.

We emphasise at the outset what this work does **not** claim. It does not introduce a new detection algorithm, it does not outperform prior work, and it does not demonstrate detection of any real ransomware sample. It is a reproduction and evaluation study whose value lies in its reproducibility and in its candour about the limits of synthetic evaluation.

The remainder of the paper is organised as follows. Section II surveys related work. Section III defines the method and the cost-sensitive objective. Section IV describes the system design and implementation. Section V details the experimental setup. Section VI presents the results, and Section VII discusses them. Section VIII states the limitations and threats to validity, and Section IX concludes and outlines future work.

II. RELATED WORK

Research on ransomware detection spans endpoint, file-system, network, and telemetry-based approaches. The method reproduced here belongs to the file-system behavioural family, which reasons about *how* files are manipulated rather than *what* they contain.

Host and file-system behaviour. CryptoLock detects ransomware by monitoring file-system activity and file entropy in real time, restoring files before encryption completes [2]. ShieldFS uses a file-system minifilter that combines low-level file-operation features with entropy to detect and transparently roll back changes [3]. Both approaches are highly effective but require kernel-level instrumentation and content inspection. UNVEIL takes a different route and detects ransomware in a sandbox by learning the sequence of file-system operations together with a screen-lock signal, reporting strong detection on a large corpus of real samples [4]. ElderRan performs automated dynamic analysis and trains classifiers on around twenty behavioural features extracted from file I/O, registry, and API-call activity [5]. These works establish that file-operation behaviour carries a usable ransomware signal, which is precisely the premise of the present study; the difference is that they rely on many features or on content, whereas we use only three aggregate counts, trading expressiveness for a very small and privacy-preserving footprint.

Network-level detection. Cabaj et al. detect crypto-ransomware from HTTP traffic characteristics within a software-defined network [6]. This complements host-side methods where traffic is observable, but it cannot see purely local file operations.

Classifier foundations. The five models compared here are standard. Support vector machines were introduced by Cortes and Vapnik [7], random forests by Breiman [8], AdaBoost by Freund and Schapire [9], and the gradient-boosted tree implementation XGBoost by Chen and Guestrin [10]; decision trees trace back to Quinlan's early work [11]. The implementation relies on the scikit-learn library [12].

Relationship to this work. The closest antecedent is the base paper [1], whose count-based, cost-sensitive, five-classifier pipeline we reproduce. That study built a virtualized small- and medium-enterprise (SME) file server, drove it with four automated user profiles, and executed five real ransomware samples (Ryuk, WannaCry, NotPetya, LockBit, and TeslaCrypt) at randomly chosen times. File operations were logged at nanosecond resolution through Fluent Bit into InfluxDB and aggregated per second. Tuning with GridSearchCV and k-fold cross-validation gave a best model — XGBoost — with a sensitivity of 91.87% (192 of 209 attack seconds detected) and sub-millisecond inference, and every model after tuning exceeded 84% sensitivity on naturally imbalanced data. Our work recreates the *offline pipeline* of that study in an independent, open code base and adds a *live testbed*, but it deliberately does not recreate the original logging stack or the real-sample execution; it substitutes a synthetic generator. To our knowledge, no openly documented reproduction of that specific server-side count-based pipeline exists, and no prior work in this family reports an explicit analysis showing that an easy synthetic evaluation set can inflate scores to 100%. Table I summarises the comparison.

TABLE I — COMPARISON WITH RELATED WORK

Work	Telemetry	Features	Live?	Real samples?
CryptoLock [2]	File-system + entropy	I/O, entropy	Yes	Yes
ShieldFS [3]	FS minifilter	I/O, entropy	Yes	Yes
UNVEIL [4]	Sandbox FS events	Op. sequences	No	Yes
ElderRan [5]	Sandbox dynamic	~20 features	No	Yes
Cabaj et al. [6]	Network (SDN)	HTTP flow stats	Yes	Yes
Base paper [1]	Server-side logs (Fluent Bit/InfluxDB)	3 counts / window	Yes	**Yes (virtualized SME)**
This work	Server-side logs (watchdog)	3 counts / window	**Yes**	**No (synthetic)**

III. METHOD

A. Features and window

File-system events are observed for three operation types: creation, rename (a move event), and deletion. Events are aggregated into fixed one-second windows. For window i, the feature vector is

$$\mathbf{x}_i = (c_i,\ r_i,\ d_i) \tag{1}$$

where c_i, r_i, and d_i are the numbers of create, rename, and delete operations in that window. Each window is labelled $y \in \{0 \text{ (benign)}, 1 \text{ (ransomware-like)}\}$. The one-second window was chosen to support near-real-time response while still smoothing over instantaneous bursts.

B. Cost-sensitive objective

The two error types are asymmetric. A false negative, *FN*, is a missed ransomware event and is by far the more dangerous outcome; a false positive, *FP*, is a false alarm, costly in analyst time but not in data loss. Following the base paper, we therefore adopt the cost matrix $C_{TP} = 0$, $C_{TN} = 0$, $C_{FP} = 1$, $C_{FN} = 10$ and score each classifier by its total cost:

$$\text{Cost} = C_{FP}\cdot FP + C_{FN}\cdot FN = FP + 10\cdot FN \tag{2}$$

This reproduces the paper's ratio in which a missed ransomware window is ten times as expensive as a false alarm.

Models are ranked by total cost, and the model with the lowest cost is selected; the same rule is applied inside the live system to display the current best model.

C. Classifiers and tuning

Five classifiers cover the main inductive biases of classical machine learning: a single Decision Tree, a Random Forest, a Support Vector Machine with an RBF kernel wrapped in a calibration layer so that it produces probabilities comparable to the ensembles, AdaBoost, and XGBoost. Each model is tuned by exhaustive grid search under stratified three-fold cross-validation, using **recall** as the optimisation score, which biases the search toward avoiding false negatives in line with the cost asymmetry. Table II lists the search spaces.

TABLE II — CLASSIFIERS AND GRID-SEARCH SPACES

Model	**Key hyperparameters searched**
Decision Tree	criterion {gini, entropy}; max_depth {3,5,7,10}; min_samples_split {2,5,10}; min_samples_leaf {1,3}
Random Forest	n_estimators {50,100,150}; criterion {gini, entropy}; max_depth {5,10,15}; min_samples_split {2,5}
SVM (RBF/linear, calibrated)	C {0.1,1,10}; kernel {rbf, linear}; gamma {scale, auto}
AdaBoost	n_estimators {50,100,150,200}; learning_rate {0.5,0.8,1.0,1.5}
XGBoost	n_estimators {50,100,150}; max_depth {3,5,7}; learning_rate {0.1,0.3,0.5}; subsample {0.7,0.9,1.0}; gamma {0,1,2}

Only the SVM uses feature standardisation, because tree-based models are invariant to monotone feature scaling.

IV. SYSTEM DESIGN AND IMPLEMENTATION

The system is realised as a single Python application. Its architecture, shown in Fig. 1, follows the method from input to output.

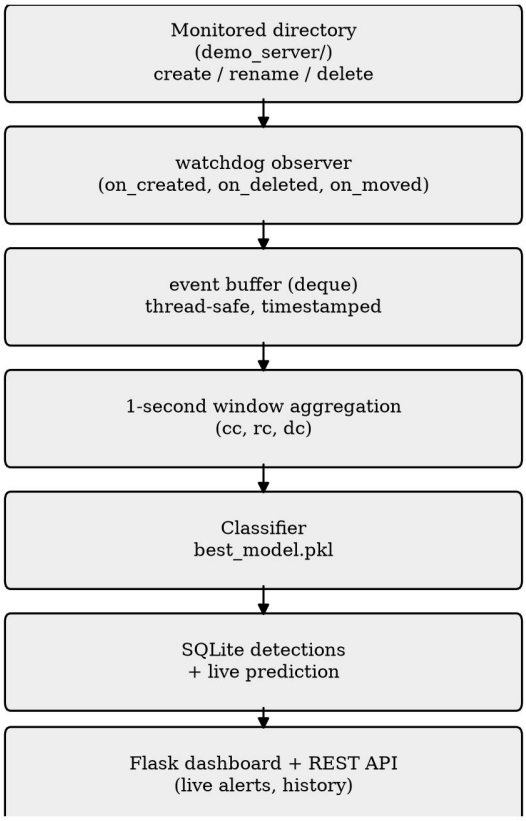


Figure 1. System architecture. File-system events from a monitored directory are captured by a `watchdog` observer, buffered in a thread-safe queue, aggregated into one-second windows, classified by the selected model, and recorded to a database; a Flask application exposes the live state to a web dashboard.

Event capture and aggregation. A recursive file-system observer subscribes to create, delete, and move events, mapping them to the create, rename, and delete operation classes. Each event is timestamped and appended to a deque guarded by a lock. A monitor loop wakes once per second, discards events older than one second, counts the three operation types, and forms the feature vector of (1). Directory events are ignored.

Classification and persistence. The monitor passes the feature vector to the selected model. If the model uses scaling, the persisted standardiser is applied first; otherwise the raw counts are used. The classifier returns a label and a confidence equal to the maximum class probability. Every window is inserted into a SQLite table with its timestamp, label, confidence, and feature counts, producing an auditable detection history.

Interfaces. A Flask application serves the dashboard and a small REST API: status and model metadata, the current window's prediction, the most recent raw events, the model comparison results, and the detection history. State-changing endpoints start and stop monitoring and trigger the two activity simulators described next.

Activity simulators. To exercise the live path without requiring malware, the testbed generates two activity patterns.

The *benign* simulator creates a handful of text documents, occasionally renames one, and less often deletes one, over a sequence of cycles. The *ransomware-like* simulator creates several dozen data files in a burst, renames them all with a `.locked` extension, and then deletes them. These simulators are demonstration instruments only; no real ransomware is used or included.

Fig. 2 shows the dashboard, which presents the current classification and confidence, the per-window create/rename/delete counts, the model comparison table, the confusion matrices, the cost comparison, and the recent detection history.

Figure 2. Live monitoring dashboard. *[SCREENSHOT REQUIRED — insert `templates/index.html` rendering.]* The figure is omitted here rather than fabricated.

Technology stack. Table III lists the software dependencies and their roles.

TABLE III — IMPLEMENTATION STACK

Component	Version (declared)	Role
Python	3.11+	Runtime
Flask	≥ 2.3	API and dashboard
scikit-learn	≥ 1.3, < 1.6	Models, tuning, metrics
XGBoost	≥ 2.0, < 3	Gradient boosting
watchdog	≥ 3.0	Real-time file events
joblib, NumPy, pandas, SciPy	—	Persistence and data handling

V. EXPERIMENTAL SETUP

A. Data

Because no labelled corpus of real server-side file-operation logs was available, the experiment uses a synthetic generator. This is a deliberate simplification of the base paper's setup: whereas that study executed real ransomware samples on a virtualized SME file server and logged them through Fluent Bit into InfluxDB, we only approximate the *shape* of the resulting count distributions. The approximation is coarse and, as shown below, far too easy. Benign windows are drawn from four behaviour profiles — a secretary, an IT administrator, a chief executive, and a mixed profile — each with low operation counts. Ransomware-like windows are drawn from five named families (Ryuk, WannaCry, LockBit, TeslaCrypt, and NotPetya), each with elevated counts. Table IV gives the ranges. The names are labels for parameter sets; they are not traces of the real malware.

TABLE IV — SYNTHETIC FEATURE RANGES (OPERATIONS PER ONE-SECOND WINDOW)

Class	create	rename	delete
Benign (secretary)	2–8	0–3	0–2
Benign (IT admin)	1–6	1–5	1–4
Benign (CEO)	0–3	0–2	0–1
Benign (mixed)	1–7	0–4	0–3
Ransomware (Ryuk)	15–55	5–25	5–20
Ransomware (WannaCry)	20–60	10–35	3–15
Ransomware (LockBit)	25–70	15–40	8–25
Ransomware (TeslaCrypt)	10–40	8–30	5–20
Ransomware (NotPetya)	30–80	10–30	10–35

Separability. The benign create count never exceeds 8, whereas the ransomware create count never falls below 10. The classes are therefore linearly separable by a threshold on a single feature. Fig. 5 (Section VI) makes this explicit. This is the single most important property of the dataset and the reason the results must be interpreted with care: the difficulty of the task, not the skill of the classifiers, is what the experiment measures.

B. Protocol

The committed artefacts correspond to a balanced dataset of 400 windows (200 benign, 200 ransomware), split 80/20 with stratification, yielding a test set of 80 windows (40 benign, 40 ransomware). Random seeds are fixed. Each classifier is tuned by three-fold stratified grid search scored by recall, refit on the training split, and evaluated once on the held-out test split. Total cost is computed with (2). Inference latency is estimated from twenty single-sample predictions, and training time is measured wall-clock. The whole pipeline is re-run in continuous integration on Python 3.11 and 3.12.

C. Environment note

The declared dependency range pins scikit-learn below 1.6, but the installed environment that produced the committed artefacts reports scikit-learn 1.6.1. The pipeline ran successfully, so the stated incompatibility between scikit-learn 1.6 and XGBoost 2.x did not reproduce here. This inconsistency is recorded as a limitation because it affects exact reproduction.

VI. RESULTS

Table V reports the test-set results; Fig. 3 compares the four threshold-based metrics, and Fig. 4 shows the total cost.

TABLE V — TEST-SET RESULTS (N = 80: 40 BENIGN, 40 RANSOMWARE), SYNTHETIC DATA

Model	Acc	Prec	Recall	F1	ROC AUC	Cost	Latency (ms)	Train (s)
Decision Tree	1.000	1.000	1.000	1.000	1.000	0	0.126	0.17
Random Forest	1.000	1.000	1.000	1.000	1.000	0	1.833	2.71
SVM (calibrated RBF)	0.988	1.000	0.975	0.987	1.000	10	0.322	0.15
AdaBoost	1.000	1.000	1.000	1.000	1.000	0	0.106	0.06
XGBoost	1.000	1.000	1.000	1.000	1.000	0	0.844	2.02

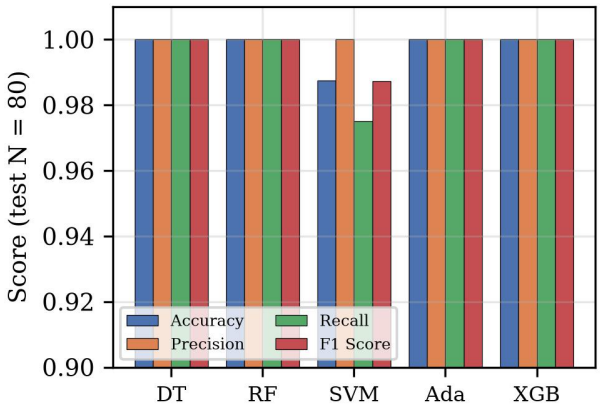


Figure 3. Accuracy, precision, recall, and F1 per classifier on the synthetic test set (N = 80). Four models are at 1.000; the SVM is marginally lower on accuracy, recall, and F1.

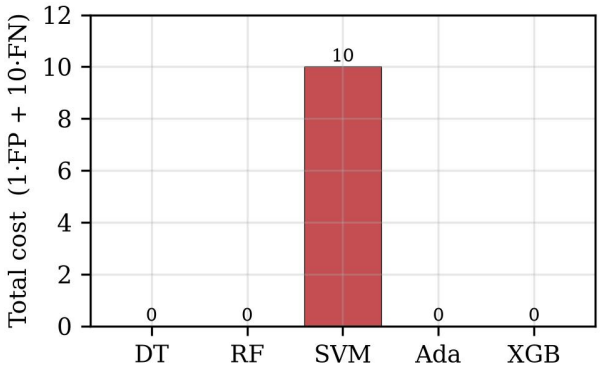


Figure 4. Total cost per classifier, $\mathrm{FP} + 10 \cdot \mathrm{FN}$. The SVM incurs cost 10 from one false negative; every other model incurs cost 0.

Four of the five classifiers separate the classes perfectly: accuracy, precision, recall, and F1 are all 1.000, ROC AUC is 1.000, and total cost is 0. The calibrated SVM is the only model that makes an error, predicting one ransomware window as benign, which lowers accuracy to 0.988, recall to 0.975, and F1 to 0.987 and raises its total cost to 10. Because four models share the minimum cost of 0, the selection rule in (2) cannot distinguish between them, and the implementation returns the first of these in its internal order, namely the Decision Tree. This is an ordering artefact rather than a measured superiority, and we do not claim that the Decision Tree is the best model.

Fig. 6 plots the ROC curves. All five curves reach the top-left corner, with area under the curve equal to 1.000; the curves differ only in the positions of intermediate thresholds, which has no practical effect at these separability levels.

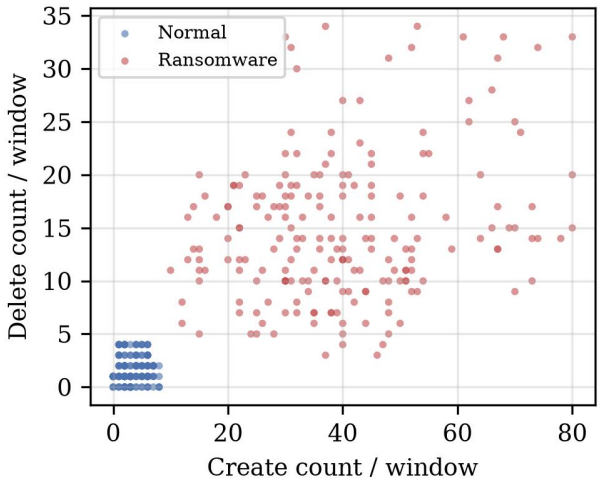


Figure 5. Create count versus delete count per window, coloured by class, on the full 400-window synthetic dataset. The classes occupy disjoint regions, confirming that the task is trivially separable.

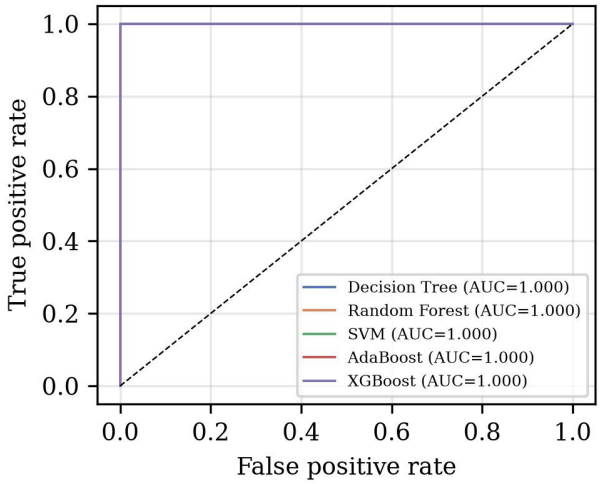


Figure 6. ROC curves for the five classifiers. All AUC values are 1.000.

Latency differences are small in absolute terms (0.106 ms for AdaBoost to 1.833 ms for Random Forest) and are dominated by the fixed one-second cadence of the monitor loop; at this window length, inference cost is negligible for every model. Training times likewise span roughly two orders of magnitude (0.06 s for AdaBoost to 2.71 s for Random Forest) but are irrelevant for a model trained once and reused.

VII. DISCUSSION

The results must be read as a statement about the *testbed and the synthetic data*, not about ransomware detection in the field. Three observations follow.

First, the near-perfect scores are explained entirely by the data. The benign and ransomware distributions in Table IV are separated by a gap on every feature, so a threshold on create count alone would achieve the same separation as any of the learned models. The experiment therefore confirms that the pipeline is wired correctly — data flows from the generator,

through tuning, to evaluation and artefacts — but it does not measure detection skill. The base paper itself provides the decisive comparison: on realistic, imbalanced SME telemetry with real ransomware samples, its best model reached 91.87% sensitivity [1]. Our synthetic run reaches 100%. Since the pipeline and model families are the same, the 8.13-point difference is attributable to the data, not to the method. Reporting only accuracy here would therefore be actively misleading, which is why we foreground the separability analysis and the cost rule.

Second, the cost-sensitive objective behaves as intended. The single false negative from the SVM is immediately visible as a jump from cost 0 to cost 10, whereas the same error would have changed accuracy by only about one percentage point. This illustrates the practical value of the cost matrix: it converts a subtle accuracy difference into an operationally meaningful one.

Third, the model selection is ambiguous by construction. When several models achieve zero cost, the tie-break has no scientific content. A more meaningful comparison would require data on which the models actually fail — that is, real telemetry — or a secondary criterion such as latency or robustness.

Taken together, these observations motivate the evaluation plan in Section IX rather than undermining the implementation, which fulfils its purpose as a reproducible baseline and a live demonstrator.

VIII. LIMITATIONS AND THREATS TO VALIDITY

We state the limitations plainly; several are severe, and none should be hidden behind neutral wording.

1. **Synthetic data (construct and external validity).** All reported numbers come from a synthetic generator whose classes are linearly separable. They do not estimate real-world performance and must never be presented as such.

2. **No real ransomware.** The live system is exercised only by a simulator, not by malware. Behavioural realism, timing, and evasion are untested.

3. **No adversarial or drift evaluation.** Rate-limited, low-and-slow encryption that stays below the count thresholds is not evaluated; such attacks would plausibly evade a pure-count detector.

4. **Single host, single directory.** The testbed monitors one directory on one machine; scalability and multi-tenant behaviour are unaddressed.

5. **Single run, no confidence intervals.** Results are point estimates from one seed. Cross-seed variability is unknown.

6. **Model-selection ambiguity.** With four models tied at zero cost, the reported "best model" is order-dependent and should not be interpreted as a ranking.

7. **Metric limitations.** ROC AUC is uninformative under perfect separation, so it cannot discriminate among the four perfect models.

8. **Reproducibility inconsistency.** The declared scikit-learn pin conflicts with the environment that produced the artefacts, and the README's summary table disagrees with the committed JSON (test-set size and latencies). The machine-generated JSON is treated as authoritative.

9. **Security of the prototype.** The dashboard API has no authentication and binds to all interfaces using a development server; it is a demonstration, not a hardened service.

Table VI summarises the principal threats and the mitigations required.

TABLE VI — THREATS TO VALIDITY AND REQUIRED MITIGATIONS

Threat	Type	Mitigation
Synthetic separable data	Construct	Re-evaluate on real logs `[REAL-WORLD RESULT REQUIRED]`
No drift/evasion study	External	Rate-limited attack scenario `[DRIFT RESULT REQUIRED]`
Single run	Statistical	≥10 seeds, report mean ± 95% CI `[CROSS-SEED CI REQUIRED]`
Model-selection tie	Internal	Secondary criterion or harder data
Environment pin mismatch	Reproducibility	Reconcile `requirements.txt` and lock versions
No auth on API	Security	Add authentication; use production server

IX. CONCLUSION AND FUTURE WORK

We have presented a reproducible implementation and live testbed for count-based ransomware detection from server-side file-operation logs. The system captures file events, aggregates them into one-second windows, classifies each window with one of five tuned models under a cost matrix that penalises missed attacks ten times more than false alarms, and exposes the results through a database and a web dashboard. On the synthetic, balanced evaluation set, four models attain perfect separation and the calibrated SVM misses one window; we have shown that these outcomes reflect the separability of the generated data rather than any real detection capability, and we have been explicit about the resulting evaluation gap.

The natural next step is to replace the synthetic generator with real, timestamped server-side file-operation logs and to repeat the evaluation with a group-aware train/test split that prevents windows from the same attack appearing on both sides. Beyond that, two extensions would convert the study from a reproduction into a genuine contribution. The first is a small set of non-count features — for example, the ratio of renames to creates, the diversity of file paths touched per window, or inter-arrival timing — evaluated by ablation so that their marginal value is measured rather than assumed. The second is a deliberate evasion experiment in which encryption is rate-limited to slip below the count threshold, which would map the boundary of the approach's competence. Until those experiments are completed, the correct reading of this work is that it provides a trustworthy, reproducible baseline and an honest account of what remains unproven.

ACKNOWLEDGMENT

[ACKNOWLEDGMENT REQUIRED — omit if none.]

No funding, sponsorship, or assistance is claimed in this draft.

REFERENCES

[1] G. Aranyi, T. Miseta, and V. Szucs, "Ransomware detection based on server-side file operation logs using machine learning," *Journal on Information Security*, vol. 8, 2026, doi: 10.1186/s13635-026-00229-7.

[2] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and drop it): Stopping ransomware attacks on user data," in *Proc. IEEE 36th Int. Conf. Distributed Computing Systems (ICDCS)*, 2016, pp. 303–312, doi: 10.1109/ICDCS.2016.46.

[3] A. Continella, A. Guagnelli, G. Zingaro, G. De Pasquale, A. Barenghi, S. Zanero, and F. Maggi, "ShieldFS: A self-healing, ransomware-aware filesystem," in *Proc. 32nd Annu. Conf. Computer Security Applications (ACSAC)*, 2016, pp. 336–347, doi: 10.1145/2991079.2991110.

[4] A. Kharaz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda, "UNVEIL: A large-scale, automated approach to detecting ransomware," in *Proc. 25th USENIX Security Symposium*, 2016, pp. 757–772.

[5] D. Sgandurra, L. Muñoz-González, R. Mohsen, and E. C. Lupu, "Automated dynamic analysis of ransomware: Benefits, limitations and use for detection," arXiv:1609.03020, 2016.

[6] K. Cabaj, M. Gregorczyk, and W. Mazurczyk, "Software-defined networking-based crypto ransomware detection using HTTP traffic characteristics," *Computers & Electrical Engineering*, vol. 66, pp. 353–368, 2018, doi: 10.1016/j.compeleceng.2017.10.012.

[7] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995, doi: 10.1007/BF00994018.

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[9] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997, doi: 10.1006/jcss.1997.1504.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[11] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986, doi: 10.1007/BF00116251.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.